GoBall - Build Environment Document

Yocto/Poky Custom Linux Distribution for Raspberry Pi 5

Ahmed Abdelghani

February 2026

Contents

1 Introduction to the Yocto Project

1.1 What is Yocto?

The Yocto Project is an open-source collaboration framework that helps developers create custom Linux distributions for embedded systems. Instead of using a general-purpose distribution like Raspbian (which includes thousands of packages you don't need), Yocto builds a minimal, purpose-built Linux image containing only what your application requires.

For GoBall, this means a Linux image that boots directly into the scoring application — no desktop environment, no unnecessary services, no login screen. Just power on and play.

1.2 Key Terminology

Term	What It Is	Analogy
Poky	The reference distribution of the Yocto Project. It includes BitBake, OpenEmbedded-Core metadata, and a default configuration.	The "starter kit" — everything you need to begin building
BitBake	The build engine (task executor). It reads recipes and executes build tasks (fetch, compile, install, package).	Like `make`, but for entire Linux distributions
OpenEmbedded (OE)	The build framework that provides the core recipes and classes. Poky bundles OE-Core.	The recipe library and build rules
Recipe (.bb)	A file that tells BitBake how to build one piece of software. Contains source URL, dependencies, compile instructions, and install rules.	A Makefile + package spec for one component
Layer (meta-*)	A directory containing related recipes, configuration, and classes. Layers are stacked — higher priority layers override lower ones.	Like plugin modules that add or customize functionality
Machine	A configuration defining the target hardware (CPU architecture, bootloader, kernel). For GoBall: `raspberrypi5`.	The "what hardware am I building for?" setting
Distro	A configuration defining the software policy (init system, package format, features). For GoBall: `goball-distro`.	The "what kind of Linux do I want?" setting
Image	The final output — a bootable filesystem image combining the kernel, root filesystem, and all installed packages.	The .img file you flash to an SD card
BSP	Board Support Package — a layer with hardware-specific kernel config, bootloader, and firmware. `meta-raspberrypi` is the RPi5 BSP.	Hardware drivers and boot configuration
bbappend	A file that modifies an existing recipe from another layer without editing the original.	A patch/override for someone else's recipe
Sysroot	A staging area containing headers and libraries for cross-compilation. BitBake populates this automatically.	The target's /usr/include and /usr/lib, but on the build host

1.3 Why Yocto for GoBall?

Approach	Boot Time	Image Size	Maintenance	Kiosk Suitability
Raspbian (full desktop)	~30s	4+ GB	Package updates can break things	Poor — must strip down
Raspbian Lite + manual config	~15s	1+ GB	Manual dependency management	Medium
Yocto (goball-image)	**~8s**	**~400 MB**	**Reproducible, version-controlled**	**Excellent — built for it**

1.4 The Build Workflow

```
[Source Code] --> [BitBake reads recipes] --> [Fetches sources]
     --> [Cross-compiles] --> [Packages] --> [Assembles rootfs]
     --> [Creates bootable image (.wic)]
```

Detailed steps:

1. **Fetch (do_fetch):** Download source code from Git, tarballs, or local files
2. **Unpack (do_unpack):** Extract archives into a work directory
3. **Patch (do_patch):** Apply any patches
4. **Configure (do_configure):** Run cmake/autotools/meson configure step
5. **Compile (do_compile):** Cross-compile for aarch64
6. **Install (do_install):** Install into a staging directory (fake root)
7. **Package (do_package):** Create RPM/DEB/IPK packages
8. **Image (do_image):** Combine all packages into a root filesystem image

2 Host System Requirements

2.1 Operating System

Requirement	Recommended	Minimum
OS	Ubuntu 22.04 LTS or 24.04 LTS	Any Linux with glibc 2.31+
Architecture	x86_64	x86_64 only

2.2 Required Packages

Install on Ubuntu:

```
sudo apt install gawk wget git diffstat unzip texinfo gcc build-essential \
    chrpath socat cpio python3 python3-pip python3-pexpect xz-utils \
    debianutils iputils-ping python3-git python3-jinja2 python3-subunit \
    zstd liblz4-tool file locales libacl1 libsdl1.2-dev xterm
```

2.3 Hardware Requirements

Resource	Minimum	Recommended	GoBall Build
CPU cores	4	8+	28 threads configured
RAM	8 GB	16+ GB	32 GB used
Disk space	50 GB	100+ GB	~80 GB (downloads + sstate + build)
Disk type	HDD	**SSD strongly recommended**	NVMe SSD

Build time estimates:

Build Type	Time (8 cores)	Time (28 cores)
First full build	~4-6 hours	~1-2 hours
Rebuild after app change	~5-10 minutes	~2-3 minutes
Rebuild with sstate cache	~10-20 minutes	~5 minutes

2.4 Target Hardware

- Raspberry Pi 5 (BCM2712, Cortex-A76, 4/8 GB RAM)
- microSD card (32 GB minimum, Class 10 / A2 recommended)
- 7” touchscreen display (2560x720 via HDMI)
- 4x IR break-beam sensors
- 2x WS2812 LED strips (144 LEDs each)
- Speaker with amplifier

3 Poky Directory Structure

The GoBall build environment lives at `/home/q/yocto/goball/poky/`:

```
poky/
+-- bitbake/              BitBake build engine
+-- meta/                 OpenEmbedded-Core recipes (base system)
+-- meta-poky/            Poky reference distro configuration
+-- meta-yocto-bsp/       Yocto reference BSP (generic hardware)
+-- meta-raspberrypi/     Raspberry Pi BSP (kernel, firmware, boot)
+-- meta-openembedded/    Community recipes (networking, multimedia, etc.)
|   +-- meta-oe/          General-purpose recipes
|   +-- meta-python/      Python packages
|   +-- meta-multimedia/  Audio/video libraries (PulseAudio, GStreamer)
|   +-- meta-networking/  NetworkManager, wpa_supplicant, etc.
+-- meta-goball/          *** Custom GoBall layer ***
+-- build/                Build output directory
|   +-- conf/             Build configuration (local.conf, bblayers.conf)
|   +-- tmp-glibc/        Build artifacts, sysroot, deployed images
+-- oe-init-build-env     Script to initialize the build environment
+-- scripts/              BitBake helper scripts
```

3.0.1 Key Directories Explained

`bitbake/` — The BitBake build engine itself. You never edit files here. It parses recipes, resolves dependencies, and orchestrates the build.

`meta/` — OpenEmbedded-Core. Contains foundational recipes for the C library (glibc), coreutils, systemd, GCC, busybox, and hundreds of base packages. This is the "operating system" layer.

`meta-raspberrypi/` — The RPi BSP layer. Provides the `raspberrypi5` machine configuration, the `linux-raspberrypi` kernel recipe, GPU firmware, VideoCore drivers, and RPi-specific device tree overlays.

`meta-openembedded/` — Community-maintained recipes. GoBall uses:

- `meta-oe`: General tools (i2c-tools, devmem2, htop)
- `meta-multimedia`: PulseAudio, ALSA plugins
- `meta-networking`: NetworkManager, wpa_supplicant
- `meta-python`: Python support (for build tools)

`meta-goball/` — The custom layer. Contains everything specific to the GoBall project. This is the only layer you edit. Covered in detail in Section 5.

`oe-init-build-env` — A shell script you source to set up the build environment. It creates the `build/` directory and sets environment variables for BitBake.

4 Build Configuration

4.1 bblayers.conf — Layer Stack

File: `build/conf/bblayers.conf`

```
POKY_BBLAYERS_CONF_VERSION = "2"

BBPATH = "${TOPDIR}"
BBFILES ?= ""

BBLAYERS ?= " \
  /home/q/yocto/goball/poky/meta \
  /home/q/yocto/goball/poky/meta-poky \
  /home/q/yocto/goball/poky/meta-yocto-bsp \
  /home/q/yocto/goball/poky/meta-raspberrypi \
  /home/q/yocto/goball/poky/meta-openembedded/meta-oe \
  /home/q/yocto/goball/poky/meta-openembedded/meta-python \
  /home/q/yocto/goball/poky/meta-openembedded/meta-multimedia \
  /home/q/yocto/goball/poky/meta-openembedded/meta-networking \
  /home/q/yocto/goball/poky/meta-goball \
  "
```

Line-by-line:

Variable	Purpose
`POKY_BBLAYERS_CONF_VERSION`	Config file format version. Must be "2" for current Poky.
`BBPATH`	BitBake search path. `${TOPDIR}` is the `build/` directory.
`BBFILES`	List of recipe files. Each layer appends to this via its `layer.conf`.
`BBLAYERS`	Ordered list of active layers. **Order matters** — later layers can override earlier ones.

Why each layer is included:

Layer	Reason
`meta`	Core OS: glibc, coreutils, systemd, gcc, cmake
`meta-poky`	Poky reference distro defaults
`meta-yocto-bsp`	Generic BSP reference (required by Poky)
`meta-raspberrypi`	RPi5 machine config, kernel, firmware, VC4 GPU
`meta-oe`	htop, nano, i2c-tools, devmem2
`meta-python`	Python dependencies for build tools
`meta-multimedia`	PulseAudio, ALSA plugins
`meta-networking`	NetworkManager, wpa_supplicant, iw
`meta-goball`	GoBall app, custom distro, device config, SDL2_mixer recipe

4.2 local.conf — Build Settings

File: `build/conf/local.conf`

```
MACHINE = "raspberrypi5"
DISTRO = "goball-distro"
INIT_MANAGER = "systemd"

# GPU / display
ENABLE_DWC2_PERIPHERAL = "0"
DISABLE_VC4GRAPHICS = "0"
VC4DTBO = "vc4-kms-v3d"

# HDMI configuration
HDMI_FORCE_HOTPLUG = "1"
RPI_EXTRA_CONFIG = " \nkernel=Image \nhdmi_group=2 \nhdmi_mode=87 \
    \nhdmi_cvt=2560 720 60 3 0 0 0 \ndisable_splash=1 \n"

# Clean boot: no logos, no boot messages, no cursor, no splash
DISABLE_SPLASH = "1"
DISABLE_RPI_BOOT_LOGO = "1"
BOOT_DELAY = "0"
BOOT_DELAY_MS = "0"
CMDLINE:append = " console=tty3 quiet loglevel=0 \
    vt.global_cursor_default=0 video=HDMI-A-2:2560x720@60D"

# Disable U-Boot (causes black screen on RPi5)
RPI_USE_U_BOOT = "0"

# PIO kernel module for WS2812 LEDs
MACHINE_EXTRA_RRECOMMENDS += "kernel-module-rp1-pio"

# Extra disk space (1GB)
IMAGE_ROOTFS_EXTRA_SPACE = "1048576"

# Accept all licenses
LICENSE_FLAGS_ACCEPTED = "commercial synaptics-killswitch"

PACKAGE_CLASSES = "package_rpm"

# Development: allow root login with password
EXTRA_IMAGE_FEATURES += "debug-tweaks"
```

4.2.1 Variable-by-Variable Explanation

Variable	Value	What It Does
`MACHINE`	`"raspberrypi5"`	Target hardware. Tells BitBake to use the RPi5 kernel, firmware, and device tree from `meta-raspberrypi`.
`DISTRO`	`"goball-distro"`	Use our custom distro policy (defined in `meta-goball/conf/distro/goball-distro.conf`). Sets systemd, PulseAudio, Wayland.
`INIT_MANAGER`	`"systemd"`	Use systemd as PID 1 (not SysVinit). Required for service management (goball.service, weston.service).

Variable	Value	What It Does
`ENABLE_DWC2_PERIPHERAL`	`"0"`	Disable USB OTG peripheral mode (not needed, prevents kernel warnings).
`DISABLE_VC4GRAPHICS`	`"0"`	Keep VC4 GPU enabled (= `0` means "don't disable"). Required for OpenGL/Wayland.
`VC4DTBO`	`"vc4-kms-v3d"`	Load the KMS/DRM device tree overlay for the GPU. Required for Weston/SDL2 display.
`HDMI_FORCE_HOTPLUG`	`"1"`	Force HDMI output even if no monitor is detected at boot. Prevents blank screen on late monitor power-on.
`RPI_EXTRA_CONFIG`	*(see below)*	Appended to `config.txt` on the boot partition. Configures HDMI resolution.
`DISABLE_SPLASH`	`"1"`	Hide the Raspberry Pi rainbow splash screen on boot.
`DISABLE_RPI_BOOT_LOGO`	`"1"`	Hide the Raspberry Pi logo from the kernel boot screen.
`BOOT_DELAY` / `BOOT_DELAY_MS`	`"0"`	No boot delay — start immediately.
`CMDLINE:append`	*(see below)*	Appended to the kernel command line.
`RPI_USE_U_BOOT`	`"0"`	Boot the kernel directly (no U-Boot). U-Boot causes a black screen on RPi5 with custom HDMI modes.
`MACHINE_EXTRA_RRECOMMENDS`	`"kernel-module-rp1-pio"`	Include the RP1 PIO kernel module. Required for WS2812 LED strip control.
`IMAGE_ROOTFS_EXTRA_SPACE`	`"1048576"`	Add 1 GB of free space to the root filesystem (for logs, updates, etc.).
`LICENSE_FLAGS_ACCEPTED`	`"commercial synaptics-killswitch"`	Accept commercial licenses for firmware blobs (WiFi, GPU).
`PACKAGE_CLASSES`	`"package_rpm"`	Use RPM format for packages (also supports deb, ipk).
`EXTRA_IMAGE_FEATURES`	`"debug-tweaks"`	Development convenience: allows root login without password, enables debug tools. **Remove for production.**

4.2.2 HDMI Configuration Explained

The `RPI_EXTRA_CONFIG` variable adds these lines to `config.txt`:

```
kernel=Image              # Boot the uncompressed kernel image
hdmi_group=2              # CEA/DMT group 2 = DMT (monitor modes)
hdmi_mode=87              # Custom mode (defined by hdmi_cvt)
hdmi_cvt=2560 720 60 3 0 0 0  # 2560x720 @ 60Hz, aspect ratio 16:9
disable_splash=1          # No rainbow screen
```

The `CMDLINE:append` adds:

```
console=tty3              # Redirect kernel messages to tty3 (not visible)
quiet                     # Suppress most kernel boot messages
loglevel=0                # Only show KERN_EMERG messages
vt.global_cursor_default=0  # Hide the blinking text cursor
video=HDMI-A-2:2560x720@60D  # Force HDMI-A-2 at 2560x720, 60Hz
```

5 The meta-goball Custom Layer

5.1 Directory Structure

```
meta-goball/
+-- conf/
|   +-- layer.conf                    Layer metadata
|   +-- distro/
|       +-- goball-distro.conf        Custom distro definition
+-- build-templates/
|   +-- local.conf.sample             Template for build/conf/local.conf
|   +-- bblayers.conf.sample          Template for build/conf/bblayers.conf
+-- recipes-core/
|   +-- images/
|   |   +-- goball-image.bb           Root filesystem image recipe
|   +-- psplash/
|   |   +-- psplash_%.bbappend        Custom boot splash screen
|   |   +-- files/
|   |       +-- psplash-goball.png    GoBall splash image (2560x720)
|   |       +-- framebuf.conf         RPi5 framebuffer fix
|   +-- dropbear/
|       +-- dropbear_%.bbappend       SSH server configuration
|       +-- dropbear/
|           +-- dropbear              PAM config
|           +-- dropbear.default      Runtime options
+-- recipes-goball/
|   +-- goball/
|       +-- goball_1.0.bb             *** Main application recipe ***
|       +-- files/
|           +-- goball.service        systemd service unit
|           +-- pulseaudio-system.service  PulseAudio system service
+-- recipes-config/
|   +-- goball-config/
|       +-- goball-config_1.0.bb      Device configuration recipe
|       +-- files/
|           +-- 99-pio.rules          PIO udev rules
|           +-- authorized_keys       SSH public keys
|           +-- Banhof.nmconnection   WiFi profile
|           +-- Ethernet.nmconnection Ethernet profile (static IP)
|           +-- rpi-eeprom-setup.sh   EEPROM configuration script
|           +-- rpi-eeprom-setup.service  First-boot EEPROM service
+-- recipes-graphics/
|   +-- wayland/
|       +-- weston-init.bbappend      Weston kiosk mode config
|       +-- weston-init/
|           +-- weston.ini            Weston compositor config
|           +-- weston.service        Weston systemd service
+-- recipes-multimedia/
|   +-- pulseaudio/
|   |   +-- pulseaudio_%.bbappend     PulseAudio access fix
|   +-- sdl2-mixer/
|       +-- libsdl2-mixer_2.8.1.bb    SDL2_mixer recipe
+-- recipes-support/
|   +-- libgpiod/
```

```
|       +-- libgpiod_1.6.5.bb         libgpiod recipe
|       +-- files/
|           +-- libgpiod-1.6.5/       Full libgpiod source tree
+-- deploy-config.conf                Deployment parameter reference
+-- setup-goball.sh                   One-command build setup script
+-- README.md                         Layer documentation
```

5.2 conf/layer.conf — Layer Registration

```
# We have a conf and classes directory, add to BBPATH
BBPATH .= ":${LAYERDIR}"

# We have recipes-* directories, add to BBFILES
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
            ${LAYERDIR}/recipes-*/*/*.bbappend"

BBFILE_COLLECTIONS += "meta-goball"
BBFILE_PATTERN_meta-goball = "^${LAYERDIR}/"
BBFILE_PRIORITY_meta-goball = "10"

LAYERDEPENDS_meta-goball = "core raspberrypi"
LAYERSERIES_COMPAT_meta-goball = "scarthgap"
```

Line-by-line:

Line	Purpose
`BBPATH .= ":${LAYERDIR}"`	Adds this layer to BitBake's search path for configuration files and classes.
`BBFILES += "..."`	Tells BitBake where to find recipes (`.bb`) and appends (`.bbappend`) in this layer. The glob `recipes-*/*/*.bb` matches all recipes in the standard directory structure.
`BBFILE_COLLECTIONS += "meta-goball"`	Registers this layer's name for conflict detection and priority resolution.
`BBFILE_PATTERN_meta-goball`	Regex matching files from this layer. Used to assign priority.
`BBFILE_PRIORITY_meta-goball = "10"`	Priority 10 (higher than default 5). Our recipes override OE-Core defaults when there's a conflict.
`LAYERDEPENDS_meta-goball = "core raspberrypi"`	This layer requires `meta` (core) and `meta-raspberrypi` to be present. BitBake errors if they're missing.
`LAYERSERIES_COMPAT_meta-goball = "scarthgap"`	Compatible with the Scarthgap release of Yocto (April 2024). Prevents accidental use with incompatible Yocto versions.

5.3 conf/distro/goball-distro.conf — Custom Distribution

```
DISTRO = "goball-distro"
DISTRO_NAME = "GoBall OS"
DISTRO_VERSION = "1.0"

DISTRO_FEATURES = "alsa pulseaudio systemd usbhost opengl wayland pam"
DISTRO_FEATURES:remove = "x11"
DISTRO_FEATURES_BACKFILL_CONSIDERED:append = " sysvinit"

VIRTUAL-RUNTIME_init_manager = "systemd"
VIRTUAL-RUNTIME_initscripts = "systemd-compat-units"
```

Line-by-line:

Variable	Purpose
`DISTRO` / `DISTRO_NAME` / `DISTRO_VERSION`	Identity strings. Appear in `/etc/os-release` on the target.
`DISTRO_FEATURES`	The set of system capabilities enabled across all packages:
– `alsa`	Low-level audio drivers (ALSA kernel interface)
– `pulseaudio`	PulseAudio sound server (provides mixing, per-app volume)
– `systemd`	systemd init system and service management
– `usbhost`	USB host support (for keyboards, USB audio, etc.)
– `opengl`	OpenGL ES support (required by Weston compositor)
– `wayland`	Wayland display protocol (replaces X11)
– `pam`	Pluggable Authentication Modules (for SSH login)
`DISTRO_FEATURES:remove = "x11"`	Explicitly remove X11. GoBall uses Wayland/Weston only.
`DISTRO_FEATURES_BACKFILL_CONSIDERED`	Tells BitBake not to automatically add `sysvinit` (we use systemd).
`VIRTUAL-RUNTIME_init_manager`	Set systemd as the init system (PID 1).
`VIRTUAL-RUNTIME_initscripts`	Use systemd compatibility scripts instead of SysVinit scripts.

5.4 recipes-core/images/goball-image.bb — The Image Recipe

This is the **top-level recipe** that defines what goes into the final SD card image.

```
SUMMARY = "GoBall Kiosk Image for Raspberry Pi 5"

IMAGE_FEATURES += "ssh-server-openssh splash"

inherit core-image extrausers

# Set root password to "123"
EXTRA_USERS_PARAMS = "usermod -p '\$5\$goball\$3Wgxx...T3U5' root;"

# Disable getty on tty1 - Weston uses it for kiosk display
disable_getty() {
    ln -sf /dev/null \
        ${IMAGE_ROOTFS}${systemd_system_unitdir}/getty@tty1.service
}
ROOTFS_POSTPROCESS_COMMAND += "disable_getty;"

IMAGE_INSTALL += " \
    goball \
    libsdl2 \
    libsdl2-mixer \
    libgpiod \
    libgpiod-tools \
    goball-config \
    weston \
    weston-init \
    mesa-megadriver \
    libegl-mesa \
    libgles2-mesa \
    libgbm \
    pulseaudio \
    pulseaudio-server \
    pulseaudio-module-alsa-sink \
    pulseaudio-module-alsa-source \
    alsa-utils \
    alsa-plugins \
    kernel-modules \
    rpi-eeprom \
    rpi-gpio \
    i2c-tools \
    devmem2 \
    nano \
    htop \
    networkmanager \
    networkmanager-nmcli \
    wpa-supplicant \
    linux-firmware-rpidistro-bcm43455 \
    iw \
"
```

5.4.1 Package Groups Explained

Core Application:

Package	Why
`goball`	The GoBall application binary + sound files
`libsdl2`	Display backend (creates Wayland window for LVGL)
`libsdl2-mixer`	Audio playback (WAV files via PulseAudio)
`libgpiod` / `libgpiod-tools`	GPIO sensor access (library + debug tools like `gpioinfo`)
`goball-config`	Device configuration (udev rules, WiFi, SSH keys, EEPROM)

Display Stack:

Package	Why
`weston`	Wayland compositor — manages the display
`weston-init`	systemd service and config for Weston
`mesa-megadriver`	OpenGL ES driver for the RPi5 GPU (V3D)
`libegl-mesa` / `libgles2-mesa`	EGL and OpenGL ES libraries (required by Weston)
`libgbm`	Generic Buffer Manager (GPU memory allocation)

Audio Stack:

Package	Why
`pulseaudio` / `pulseaudio-server`	Sound server (mixing, routing)
`pulseaudio-module-alsa-sink/source`	ALSA backend modules for PulseAudio
`alsa-utils` / `alsa-plugins`	Low-level audio tools and ALSA compatibility

System Utilities:

Package	Why
`kernel-modules`	All kernel modules (GPIO, PIO, audio, networking)
`rpi-eeprom`	RPi5 EEPROM firmware update tool
`rpi-gpio`	GPIO utility scripts
`i2c-tools` / `devmem2`	Hardware debugging tools
`nano` / `htop`	Basic text editor and process monitor (dev convenience)

Networking:

Package	Why
`networkmanager` / `networkmanager-nmcli`	Network management (WiFi, Ethernet)
`wpa-supplicant`	WPA/WPA2 WiFi authentication
`linux-firmware-rpidistro-bcm43455`	WiFi firmware for the RPi5's Broadcom chip
`iw`	WiFi diagnostic tool

5.4.2 Special Configuration

`IMAGE_FEATURES += "ssh-server-openssh splash"`

- `ssh-server-openssh`: Installs OpenSSH server for remote access
- `splash`: Enables the psplash boot splash screen

`inherit core-image extrausers`

- `core-image`: Base class for building root filesystem images
- `extrausers`: Allows setting user passwords at build time

`disable_getty()`: Creates a symlink from `getty@tty1.service` to `/dev/null`, effectively disabling the text login prompt on tty1. This is necessary because Weston takes over tty1 for display output.

5.5 recipes-goball/goball/goball_1.0.bb — The Application Recipe

This is the most important recipe — it builds the GoBall application from source.

```
SUMMARY = "GoBall Mini Golf Scoring System"
DESCRIPTION = "LVGL-based mini golf scoring application with \
    GPIO sensors and LED support"
LICENSE = "MIT"
LIC_FILES_CHKSUM = \
    "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"

DEPENDS = "libsdl2 libsdl2-mixer libgpiod"

SRC_URI = "git://github.com/aabdelghani/GoBall.git;\
    protocol=https;branch=master \
           file://goball.service \
           file://pulseaudio-system.service"
SRCREV = "${AUTOREV}"

S = "${WORKDIR}/git"

inherit cmake systemd pkgconfig

EXTRA_OECMAKE = "-DCMAKE_BUILD_TYPE=Debug \
    -DYOCTO_BUILD=ON \
    -DSOUND_DIR_PATH=/opt/goball/sounds/"

SYSTEMD_SERVICE:${PN} = "goball.service \
    pulseaudio-system.service"
SYSTEMD_AUTO_ENABLE = "enable"

do_install() {
    install -d ${D}${bindir}
    install -m 0755 ${B}/SquareLine_Project ${D}${bindir}/goball

    install -d ${D}/opt/goball/sounds
    for f in $(find ${S}/modules/game_sounds -name '*.wav'); do
        install -m 0644 "$f" ${D}/opt/goball/sounds/
    done

    install -d ${D}${systemd_system_unitdir}
    install -m 0644 ${WORKDIR}/goball.service \
        ${D}${systemd_system_unitdir}/goball.service
    install -m 0644 ${WORKDIR}/pulseaudio-system.service \
        ${D}${systemd_system_unitdir}/pulseaudio-system.service
}

FILES:${PN} += "/opt/goball /opt/goball/sounds"
```

5.5.1 Line-by-Line Explanation

Metadata:

Variable	Purpose
`SUMMARY` / `DESCRIPTION`	Human-readable package description
`LICENSE = "MIT"`	The project's open-source license
`LIC_FILES_CHKSUM`	MD5 checksum of the license file (BitBake verifies this to ensure the license hasn't changed)

Dependencies:

```
DEPENDS = "libsdl2 libsdl2-mixer libgpiod"
```

Build-time dependencies. BitBake ensures these are compiled and their headers/libraries are available in the sysroot before building GoBall.

Source:

```
SRC_URI = "git://github.com/aabdelghani/GoBall.git;protocol=https;branch=master \
           file://goball.service \
           file://pulseaudio-system.service"
SRCREV = "${AUTOREV}"
```

- Fetches the GoBall source from GitHub (master branch)
- Also includes two local files from the `files/` directory
- `AUTOREV` means "always use the latest commit" (good for development; pin to a specific SHA for production)

Source directory:

```
S = "${WORKDIR}/git"
```

After `do_fetch` and `do_unpack`, the Git clone is at `${WORKDIR}/git`. This tells BitBake where to find the source code.

Build system:

```
inherit cmake systemd pkgconfig
```

- `cmake`: Use CMake for configure/compile (calls `cmake` and `make`)
- `systemd`: Automatically enables/disables systemd services
- `pkgconfig`: Sets up pkg-config for finding libraries

CMake flags:

```
EXTRA_OECMAKE = "-DCMAKE_BUILD_TYPE=Debug \
    -DYOCTO_BUILD=ON \
    -DSOUND_DIR_PATH=/opt/goball/sounds/"
```

Flag	Effect in CMakeLists.txt
`-DCMAKE_BUILD_TYPE=Debug`	Debug build with level 4 logging
`-DYOCTO_BUILD=ON`	Skips local SDL2 paths and toolchain file (Yocto provides these)
`-DSOUND_DIR_PATH=/opt/goball/sounds/`	Overrides the sound directory path (compiled into the binary)

systemd integration:

```
SYSTEMD_SERVICE:${PN} = "goball.service pulseaudio-system.service"
SYSTEMD_AUTO_ENABLE = "enable"
```

Registers both services with systemd. `enable` means they start automatically on boot.

Custom install (do_install):

The `do_install()` function runs after compilation and places files into the staging directory (`${D}` = destination root):

1. **Binary:** Copies `SquareLine_Project` binary to `/usr/bin/goball`
2. **Sounds:** Copies all `.wav` files from the source tree to `/opt/goball/sounds/`
3. **Services:** Installs both systemd service unit files

Package files:

```
FILES:${PN} += "/opt/goball /opt/goball/sounds"
```

Tells the packager to include `/opt/goball/` in the `goball` package (by default, only standard paths like `/usr/bin` are included automatically).

5.5.2 goball.service — Application Service Unit

```
[Unit]
Description=GoBall Mini Golf Scoring System
After=weston.service sound.target pulseaudio-system.service
Wants=pulseaudio-system.service
Requires=weston.service

[Service]
Type=simple
Environment="SDL_VIDEODRIVER=wayland"
Environment="SDL_VIDEO_GL_DRIVER=libGLESv2.so.2"
Environment="SDL_AUDIODRIVER=pulseaudio"
Environment="WAYLAND_DISPLAY=wayland-1"
Environment="XDG_RUNTIME_DIR=/run/weston"
Environment="PULSE_SERVER=unix:/run/pulse/native"
ExecStartPre=/bin/sleep 2
ExecStart=/usr/bin/goball
Restart=on-failure
RestartSec=5
User=root
SupplementaryGroups=pulse-access

[Install]
WantedBy=multi-user.target
```

Directive	Purpose
`After=weston.service`	Start after the display compositor is running
`Requires=weston.service`	Hard dependency — if Weston fails, GoBall doesn't start
`Wants=pulseaudio-system.service`	Soft dependency — GoBall runs even if audio fails
`SDL_VIDEODRIVER=wayland`	Tell SDL2 to use Wayland (not X11 or fbdev)
`SDL_AUDIODRIVER=pulseaudio`	Tell SDL2 to use PulseAudio for audio output
`WAYLAND_DISPLAY=wayland-1`	Connect to Weston's Wayland socket
`XDG_RUNTIME_DIR=/run/weston`	Runtime directory where Weston creates its socket
`PULSE_SERVER=unix:/run/pulse/native`	PulseAudio socket path (system mode)
`ExecStartPre=/bin/sleep 2`	Wait 2 seconds for Weston to fully initialize

Directive	Purpose
`Restart=on-failure`	Auto-restart if the app crashes
`SupplementaryGroups=pulse-access`	Grant access to the PulseAudio socket

5.5.3 pulseaudio-system.service — Audio Service

```
[Unit]
Description=PulseAudio System Mode
After=sound.target

[Service]
Type=notify
ExecStart=/usr/bin/pulseaudio --system --disallow-exit \
    --disallow-module-loading --log-target=journal
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

PulseAudio runs in **system mode** (not per-user) because there's no login session — Weston runs as root directly. The `--disallow-exit` flag prevents PulseAudio from shutting down when no clients are connected.

5.6 recipes-config/goball-config/goball-config_1.0.bb — Device Configuration

```
SUMMARY = "GoBall device configuration"
LICENSE = "MIT"
LIC_FILES_CHKSUM = \
    "file://${COMMON_LICENSE_DIR}/MIT;md5=0835ade698e0bcf8506ecda2f7b4f302"

SRC_URI = "file://99-pio.rules \
           file://Banhof.nmconnection \
           file://Ethernet.nmconnection \
           file://authorized_keys \
           file://rpi-eeprom-setup.sh \
           file://rpi-eeprom-setup.service"

inherit useradd systemd

SYSTEMD_SERVICE:${PN} = "rpi-eeprom-setup.service"
SYSTEMD_AUTO_ENABLE = "enable"

USERADD_PACKAGES = "${PN}"
GROUPADD_PARAM:${PN} = "-r pulse-access"

do_install() {
    # PIO udev rules
    install -d ${D}${sysconfdir}/udev/rules.d
    install -m 0644 ${WORKDIR}/99-pio.rules \
        ${D}${sysconfdir}/udev/rules.d/

    # WiFi auto-connect profile (NetworkManager)
    install -d ${D}${sysconfdir}/NetworkManager/system-connections
    install -m 0600 ${WORKDIR}/Banhof.nmconnection \
        ${D}${sysconfdir}/NetworkManager/system-connections/
    install -m 0600 ${WORKDIR}/Ethernet.nmconnection \
        ${D}${sysconfdir}/NetworkManager/system-connections/

    # SSH authorized keys for root
    install -d ${D}/root/.ssh
    install -m 0600 ${WORKDIR}/authorized_keys \
        ${D}/root/.ssh/authorized_keys
    chmod 700 ${D}/root/.ssh

    # EEPROM setup script + service
    install -d ${D}${bindir}
    install -m 0755 ${WORKDIR}/rpi-eeprom-setup.sh \
        ${D}${bindir}/rpi-eeprom-setup.sh
    install -d ${D}${systemd_system_unitdir}
    install -m 0644 ${WORKDIR}/rpi-eeprom-setup.service \
        ${D}${systemd_system_unitdir}/
}

RDEPENDS:${PN} += "rpi-eeprom"
FILES:${PN} += "/root/.ssh"
```

This recipe installs six configuration files:

5.6.1 99-pio.rules — PIO Device Permissions

```
SUBSYSTEM=="*-pio", GROUP="gpio", MODE="0660"
```

This udev rule gives the `gpio` group read/write access to PIO devices (`/dev/pio0`, etc.). Without this, only root can access PIO, and the LED controller initialization fails.

5.6.2 WiFi Profile (Banhof.nmconnection)

```
[connection]
id=Banhof
type=wifi
autoconnect=true
autoconnect-priority=100

[wifi]
ssid=Banhof
mode=infrastructure

[wifi-security]
key-mgmt=wpa-psk
psk=CHANGEME

[ipv4]
method=auto
```

NetworkManager auto-connects to this WiFi network on boot. The `.sample` file is versioned in Git; the actual file with the real password is `.gitignore`d.

5.6.3 Ethernet Profile (Ethernet.nmconnection)

```
[connection]
id=Ethernet
type=ethernet
autoconnect=true
autoconnect-priority=200

[ipv4]
method=manual
addresses=10.0.0.2/24
```

Static IP (10.0.0.2) for direct laptop-to-Pi Ethernet connection. Priority 200 > WiFi's 100, so Ethernet is preferred when both are connected.

5.6.4 EEPROM Setup Script (rpi-eeprom-setup.sh)

Runs once on first boot to configure the RPi5 EEPROM:

- Sets `DISPLAY_DIAGNOSTIC=0` — hides the EEPROM bootloader diagnostic screen
- Sets `NET_INSTALL_AT_POWER_ON=0` — disables the "install OS from network" prompt
- Creates a flag file (`/var/lib/goball/eeprom-configured`) to skip on subsequent boots

5.7 recipes-multimedia/ — Audio Libraries

5.7.1 libsdl2-mixer_2.8.1.bb — SDL2_mixer Recipe

```
SUMMARY = "SDL2 multi-channel audio mixer library"
LICENSE = "Zlib"
LIC_FILES_CHKSUM = \
    "file://LICENSE.txt;md5=fbb0010b2f7cf6e8a13bcac1ef4d2455"

DEPENDS = "libsdl2"

SRC_URI = "https://github.com/libsdl-org/SDL_mixer/releases/\
    download/release-${PV}/SDL2_mixer-${PV}.tar.gz"
SRC_URI[sha256sum] = "cb760211b056bfe44f4a1e180cc7cb201137e4d..."

S = "${WORKDIR}/SDL2_mixer-${PV}"

inherit cmake pkgconfig

EXTRA_OECMAKE = " \
    -DSDL2MIXER_WAVE=ON \
    -DSDL2MIXER_MP3=OFF \
    -DSDL2MIXER_FLAC=OFF \
    -DSDL2MIXER_MOD=OFF \
    -DSDL2MIXER_MIDI=OFF \
    -DSDL2MIXER_OPUS=OFF \
    -DSDL2MIXER_WAVPACK=OFF \
    -DSDL2MIXER_VORBIS=OFF \
"

PROVIDES = "libsdl2-mixer"
RPROVIDES:${PN} = "libsdl2-mixer"
```

Why a custom recipe? The default OE-Core recipe for SDL2_mixer may not exist or may pull in heavy dependencies (libvorbis, libflac, libmpg123). This recipe enables **only WAV support** — the only format GoBall uses — keeping the image small and dependency-free.

5.7.2 pulseaudio_%.bbappend — PulseAudio Fix

```
do_install:append() {
    sed -i 's/load-module module-native-protocol-unix.*/\
        load-module module-native-protocol-unix auth-anonymous=1/' \
        ${D}${sysconfdir}/pulse/system.pa
}
```

This modifies PulseAudio's `system.pa` configuration to allow anonymous local connections. Without this, the GoBall process (running as root) cannot connect to PulseAudio's UNIX socket in system mode.

5.8 recipes-graphics/ — Display Configuration

5.8.1 weston-init.bbappend — Kiosk Mode

```
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"
PACKAGECONFIG:append = " no-idle-timeout"
```

Disables Weston's idle timeout so the screen never turns off.

5.8.2 weston.ini — Compositor Configuration

```
[core]
shell=kiosk-shell.so
require-input=false
idle-time=0

[output]
name=HDMI-A-1
mode=2560x720

[output]
name=HDMI-A-2
mode=2560x720
```

Setting	Purpose
`shell=kiosk-shell.so`	Kiosk shell — single fullscreen app, no window decorations, no taskbar
`require-input=false`	Start even without keyboard/mouse connected
`idle-time=0`	Never blank the screen
Output sections	Configure both HDMI ports to 2560x720 (whichever port is used)

5.8.3 weston.service — Compositor Service

```
[Unit]
Description=Weston Wayland Compositor (Kiosk Mode)
After=systemd-user-sessions.service dbus.socket
ConditionPathExists=/dev/tty0

[Service]
Type=notify
ExecStartPre=/bin/sleep 5
ExecStartPre=/bin/sh -c 'mkdir -p /run/weston && chmod 0700 /run/weston'
Environment="XDG_RUNTIME_DIR=/run/weston"
ExecStart=/usr/bin/weston --modules=systemd-notify.so
User=root
TTYPath=/dev/tty1
```

Weston starts 5 seconds after boot (giving the GPU time to initialize), runs on tty1 as root, and creates its runtime directory at `/run/weston`.

5.9 recipes-core/psplash/ — Boot Splash

5.9.1 psplash_%.bbappend

```
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
SPLASH_IMAGES:raspberrypi5 = \
    "file://psplash-goball.png;outsuffix=default"
SRC_URI:append:raspberrypi5 = " file://framebuf.conf"
```

Replaces the default Poky splash screen with a custom GoBall-branded PNG (2560x720). The `:raspberrypi5` override ensures this only applies when building for the RPi5 machine.

The `framebuf.conf` file is a systemd drop-in that fixes a framebuffer device dependency issue specific to the RPi5 (the device path differs from earlier models).

5.10 recipes-support/libgpiod/ — GPIO Library

5.10.1 libgpiod_1.6.5.bb

```
SUMMARY = "C library and tools for interacting with the \
    linux GPIO character device"
LICENSE = "LGPL-2.1-or-later"
LIC_FILES_CHKSUM = \
    "file://COPYING;md5=2caced0b25dfefd4c601d92bd15116de"

SRC_URI = "file://${BP}"
S = "${WORKDIR}/${BP}"

DEPENDS = "autoconf-archive-native"
inherit autotools pkgconfig

PACKAGECONFIG[tools] = "--enable-tools,--disable-tools"
PACKAGECONFIG = "tools"

PROVIDES = "libgpiod"
RPROVIDES:${PN} = "libgpiod"

PACKAGES =+ "${PN}-tools"
FILES:${PN}-tools = "${bindir}/*"
```

Why a custom recipe? GoBall requires libgpiod v1.6.x (the v1 API with `gpiod_line_event_wait_bulk()`). The OE-Core recipe provides v2.x which has an incompatible API. This recipe bundles the full libgpiod 1.6.5 source tree in `files/libgpiod-1.6.5/` and builds it using autotools.

The `${BP}` variable expands to `libgpiod-1.6.5` (base package name + version), matching the local source directory.

5.11 recipes-core/dropbear/ — SSH Configuration

5.11.1 dropbear_%.bbappend

```
FILESEXTRAPATHS:prepend := "${THISDIR}/${PN}:"
PAM_SRC_URI = "file://dropbear"
PACKAGECONFIG:remove = "pam"
```

Disables PAM for the Dropbear SSH server. PAM on RPi5 has compatibility issues that prevent root login. The fallback SSH server is OpenSSH (added via `IMAGE_FEATURES += "ssh-server-openssh"`), so Dropbear is effectively unused but this append prevents build errors.

6 Kernel Configuration

6.1 Kernel Recipe

The kernel comes from `meta-raspberrypi` via the `linux-raspberrypi` recipe. It builds the official Raspberry Pi kernel with RPi5-specific patches and device tree overlays.

6.2 Required Kernel Features

Feature	Config	Why
GPIO character device	`CONFIG_GPIO_CDEV`	libgpiod accesses GPIOs via `/dev/gpiochip0`
RP1 PIO module	`CONFIG_RP1_PIO`	WS2812 LED control via PIO hardware
V3D GPU driver	`CONFIG_DRM_V3D`	OpenGL ES for Weston compositor
KMS/DRM	`CONFIG_DRM`	Display output via kernel mode setting
ALSA	`CONFIG_SND`	Audio hardware access
USB host	`CONFIG_USB`	USB peripherals
WiFi (brcmfmac)	`CONFIG_BRCMFMAC`	RPi5 WiFi chip driver

The PIO kernel module is explicitly included via `local.conf`:

```
MACHINE_EXTRA_RRECOMMENDS += "kernel-module-rp1-pio"
```

This ensures the `rp1-pio` module is packaged and loaded at boot, allowing GoBall's `piolib` to open `/dev/pio0` for LED control.

7 Building the Image

7.1 Quick Start (First Time)

```
# 1. Clone and set up everything
git clone https://github.com/aabdelghani/meta-goball.git
./meta-goball/setup-goball.sh ~/yocto/goball

# 2. Enter the build environment
cd ~/yocto/goball/poky
source oe-init-build-env build

# 3. Edit WiFi credentials
nano meta-goball/recipes-config/goball-config/files/Banhof.nmconnection

# 4. Add your SSH public key
cat ~/.ssh/id_ed25519.pub > \
    meta-goball/recipes-config/goball-config/files/authorized_keys

# 5. Build the image
bitbake goball-image

# 6. Flash to SD card
sudo bmaptool copy \
    tmp-glibc/deploy/images/raspberrypi5/\
    goball-image-raspberrypi5.rootfs.wic.bz2 \
    /dev/sdX
```

7.2 What setup-goball.sh Does

The setup script automates the initial environment:

1. Clones Poky (Scarthgap branch) if not present
2. Clones `meta-raspberrypi` and `meta-openembedded` layers
3. Clones `meta-goball` from GitHub
4. Sources `oe-init-build-env` to create the `build/` directory
5. Copies template configs (`local.conf.sample`, `bblayers.conf.sample`) to `build/conf/`
6. Replaces `##POKYDIR##` placeholder with the actual Poky path
7. Creates `.sample` copies of sensitive files (WiFi credentials, SSH keys)

7.3 Rebuilding After Code Changes

When you push code changes to the GoBall GitHub repo:

```
cd ~/yocto/goball/poky
source oe-init-build-env build

# Clean the old build and rebuild
bitbake goball -c cleansstate
bitbake goball-image
```

`cleansstate` removes all cached build artifacts for the `goball` recipe, forcing a fresh clone and rebuild. Other packages (SDL2, kernel, etc.) remain cached.

7.4 Useful BitBake Commands

Command	Purpose
`bitbake goball-image`	Build the full image
`bitbake goball`	Build only the GoBall package
`bitbake goball -c cleansstate`	Clean GoBall build cache
`bitbake goball -c devshell`	Open a shell in the build environment
`bitbake -e goball | grep ^S=`	Show the source directory path
`bitbake -g goball-image`	Generate dependency graph
`bitbake -s | grep sdl`	Search available recipes

8 Deploying to Raspberry Pi 5

8.1 Flashing the SD Card

```
# Find your SD card device
lsblk

# Flash (bmaptool is faster than dd)
sudo bmaptool copy \
    tmp-glibc/deploy/images/raspberrypi5/\
    goball-image-raspberrypi5.rootfs.wic.bz2 \
    /dev/sdX

# Or with dd (slower)
bzcat tmp-glibc/deploy/images/raspberrypi5/\
    goball-image-raspberrypi5.rootfs.wic.bz2 \
    | sudo dd of=/dev/sdX bs=4M status=progress
```

8.2 First Boot

1. Insert the SD card into the RPi5
2. Connect HDMI display, sensors, and power
3. The boot sequence is:
    - RPi5 EEPROM bootloader (hidden after first EEPROM config)
    - Custom psplash boot screen (GoBall branding)
    - Weston compositor starts (kiosk shell)
    - PulseAudio starts (system mode)
    - GoBall starts automatically

8.3 Updating Just the Application

To update GoBall without reflashing:

```
# Cross-compile on host
cd /home/q/1Projects/GoBall/SquareLine_Project
cmake -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
make -C build -j$(nproc)

# Copy to Pi
scp build/SquareLine_Project root@10.0.0.2:/usr/bin/goball

# Restart on Pi
ssh root@10.0.0.2 systemctl restart goball
```

8.4 Boot Service Chain

```
power on
  --> EEPROM bootloader (RPi5 firmware)
  --> kernel (direct boot, no U-Boot)
  --> systemd (PID 1)
    --> psplash (boot splash)
    --> rpi-eeprom-setup (one-shot, first boot only)
    --> pulseaudio-system.service (audio server)
    --> weston.service (display compositor)
      --> 5s delay (GPU init)
```

```
--> goball.service (the application)
  --> 2s delay (wait for Wayland socket)
  --> /usr/bin/goball
```

9 Development Workflow

9.1 The YOCTO_BUILD CMake Flag

When building inside Yocto, the recipe passes `-DYOCTO_BUILD=ON`. In `CMakeLists.txt`:

```
if(NOT YOCTO_BUILD)
    set(CMAKE_TOOLCHAIN_FILE toolchain-aarch64.cmake)
endif()

if(NOT YOCTO_BUILD)
    set(SDL_FOLDER sdl2-dev-rpi64)
    include_directories(${PROJECT_SOURCE_DIR}/${SDL_FOLDER}/include)
    target_link_directories(${PROJECT_NAME} PRIVATE
        ${PROJECT_SOURCE_DIR}/${SDL_FOLDER}/lib
        ${PROJECT_SOURCE_DIR}/rpi5-sysroot/usr/lib/aarch64-linux-gnu)
endif()
```

This flag:

- Skips the local `toolchain-aarch64.cmake` (Yocto provides its own cross-compiler)
- Skips the bundled SDL2 headers/libraries (Yocto provides these via sysroot)
- Allows the same `CMakeLists.txt` to work for both Yocto builds and local cross-compilation

9.2 Adding a New Package to the Image

1. Find the recipe: `bitbake -s | grep <package>`
2. Add to `goball-image.bb`: `IMAGE_INSTALL += "package-name"`
3. Rebuild: `bitbake goball-image`

If no recipe exists, create one in `meta-goball/recipes-<category>/<name>/<name>_<version>.bb`.

9.3 Modifying Kernel Configuration

To add a kernel config option:

```
# 1. Start a kernel devshell
bitbake virtual/kernel -c devshell

# 2. Run menuconfig
make menuconfig

# 3. Save and exit

# 4. Copy the changed config
# Create a .cfg fragment in meta-goball with just the changed options
```

Or create a `.cfg` fragment file and add it via a `linux-raspberrypi_%.bbappend`.

10 Troubleshooting

10.1 Common Build Errors

Error	Cause	Fix
`Nothing PROVIDES 'libsdl2-mixer'`	Missing recipe	Verify `meta-goball` is in `bblayers.conf`
`do_fetch failed`	Network issue or wrong SRCREV	Check internet; use `SRCREV = "${AUTOREV}"` for dev
`QA Issue: No GNU_HASH`	Missing compiler flags	Add `-Wl,--hash-style=gnu` to LDFLAGS
`sstate cache miss`	First build or different machine	Normal — takes longer on first build
`Disk space`	/tmp or build dir full	Set `DL_DIR` and `SSTATE_DIR` to larger disk

10.2 Runtime Issues

Symptom	Cause	Fix
Black screen	U-Boot enabled	Set `RPI_USE_U_BOOT = "0"` in local.conf
No sound	PulseAudio auth	Check `pulseaudio_%.bbappend` has `auth-anonymous=1`
No WiFi	Missing firmware	Verify `linux-firmware-rpidistro-bcm43455` in IMAGE_INSTALL
No LEDs	PIO permissions	Check `99-pio.rules` installed, `kernel-module-rp1-pio` loaded
App crashes	Missing Wayland socket	Check `weston.service` runs before `goball.service`

10.3 Debugging Commands (on target)

```
# Check service status
systemctl status goball
systemctl status weston
systemctl status pulseaudio-system

# View application logs
journalctl -u goball -f

# Check GPIO
gpioinfo

# Check PIO
ls -la /dev/pio*

# Check audio
```

```
pactl info
aplay -l

# Check display
weston-info

# Rebuild and deploy quickly
# (on host)
bitbake goball -c cleansstate && bitbake goball
scp build/tmp-glibc/work/*/goball/*/image/usr/bin/goball \
    root@10.0.0.2:/usr/bin/goball
ssh root@10.0.0.2 systemctl restart goball
```

11 Appendix A: Deployment Configuration Reference

The file `meta-goball/deploy-config.conf` contains a complete reference of every parameter that must be reviewed when deploying to a new machine. Parameters are marked as:

- **[REBUILD]** — Requires a Yocto rebuild and SD card reflash
- **[CODE]** — Requires a code change, recipe clean, and rebuild
- **[RUNTIME]** — Can be changed on the running device via SSH

Key deployment parameters:

Parameter	Default	Type	File
WiFi SSID/Password	Banhof/CHANGEME	REBUILD	Banhof.nmconnection
Ethernet IP	10.0.0.2/24	REBUILD	Ethernet.nmconnection
HDMI Port	HDMI-A-2	REBUILD	local.conf, weston.ini
Display Resolution	2560x720	REBUILD+CODE	local.conf, weston.ini, main.c
GPIO Sensor Pins	17,26,27,24	CODE	gpio_event.h
LED Strip GPIOs	3, 2	CODE	main.c
LED Pixel Count	144	CODE	led_logic_event.h
Sound Directory	/opt/goball/sounds/	REBUILD	goball_1.0.bb
Root Password	123	REBUILD	goball-image.bb
SSH Keys	(your key)	REBUILD	authorized_keys
Build Type	Debug	REBUILD	goball_1.0.bb

12 Appendix B: Yocto/Poky Version Information

Component	Version	Branch
Poky	5.0	Scarthgap
BitBake	2.8	Scarthgap
GCC (cross)	14.x	Provided by OE-Core
Linux kernel	6.6.x	linux-raspberrypi
meta-raspberrypi	Scarthgap	Scarthgap
meta-openembedded	Scarthgap	Scarthgap